WebRTC: Theory to Practice (mediasoup + LiveKit with Express)

Oct 18, 2026 · @Subhamoy Ghosh

How to use this guide

This guide takes you from "what is a peer connection?" to two working group-video servers: one you assemble yourself with mediasoup, one you drive through LiveKit, both behind an Express.js backend.

The practical parts are written as **steps you code yourself**. Every step opens with its aim, its inputs and outputs, and a check that tells you it worked. Short skeletons show the shape of the API, not a finished app, so you write the glue and learn it.

Part	What you learn	What you end with
1. Theory (sections 1 to 5)	Signaling, SDP, ICE, DTLS-SRTP, RTP, topologies	A mental model you can debug with
2. Setup (section 6)	Project layout, getUserMedia, secure contexts	Express server, camera preview in the browser
3. mediasoup (section 7)	Workers, routers, transports, producers, consumers	Multi-user video room on your own SFU
4. LiveKit (section 8)	Tokens, rooms, tracks, webhooks, server API	Multi-user video room on a managed-style SFU
5. Wrap-up (section 9)	Comparison, TURN, ports, scaling, debugging	A decision table and exercises

Prerequisites

- Node.js 20 or newer and npm. Everything here is JavaScript on both the server and the browser.
- Basic Express.js and Socket.IO (events, rooms) and `async/await`.
- A Chromium-based browser or Firefox. WebRTC needs `https://` or `http://localhost`, so test locally on `localhost`.

- Linux, macOS or WSL2 for mediasoup (it compiles a native worker). Windows native works but needs build tools. LiveKit needs its server binary or Docker.

Step format used in sections 6 to 8

1. **Aim**: what you are building and why.
2. **Input / Output**: what goes in and what comes out (function arguments, socket events, JSON).
3. **Skeleton**: the key API calls, with gaps for you to fill.
4. **Check**: what to see in the console or browser to know it works.

Software versions move quickly in this space. The API names here follow mediasoup 3.x, mediasoup-client 3.x, LiveKit server SDK 2.x and livekit-client 2.x. If a call fails with "not a function", check the changelog of your installed version first.

1. What WebRTC is

WebRTC (Web Real-Time Communication) lets browsers and apps send audio, video and arbitrary data directly to each other with sub-second latency, without plugins. It is a set of browser APIs plus a set of network protocols, standardised by the W3C (the JavaScript APIs) and the IETF (the protocols).

The single most important idea: **WebRTC standardises how media and data move, but not how peers find each other.** Finding each other and agreeing on parameters is called *signaling*, and you build it yourself (usually with WebSockets or Socket.IO). That is why every WebRTC app has an Express/Node server even when the media is "peer to peer".

The three browser APIs

API	Job	You call it when
`MediaStream` / `getUserMedia`	Capture camera, microphone or screen into tracks	You need local media
`RTCPeerConnection`	Connect to another peer: negotiate, encrypt, send and receive tracks	Always, for audio and video
`RTCDataChannel`	Send arbitrary bytes or text over the same connection	Chat, game state, file transfer

Vocabulary you must own

- **Track**: one stream of audio or video (`MediaStreamTrack`). A `MediaStream` is just a bag of tracks.
- **Peer**: one endpoint of a connection. In an SFU setup, the server is a peer too.
- **Signaling**: your own channel for exchanging connection setup messages (SDP, ICE candidates).
- **SDP**: a text description of what media each side can send and receive and how.
- **ICE candidate**: one possible network address (IP + port + protocol) where a peer might be reached.
- **STUN / TURN**: helper servers for discovering your public address and for relaying traffic when direct paths fail.
- **SFU**: Selective Forwarding Unit, a server that receives media from each participant and forwards it to the others.

The life of a call, end to end

1. Each side captures media with `getUserMedia` and creates an `RTCPeerConnection` .
2. Side A calls `createOffer()` , sets it as its local description, and sends the SDP offer through signaling.
3. Side B sets it as its remote description, calls `createAnswer()` , sets that locally, and sends the answer back.
4. Meanwhile both sides gather ICE candidates and trickle them to each other through signaling.
5. ICE tests candidate pairs and picks one that works. A DTLS handshake follows over that path to exchange keys.
6. Encrypted media flows as SRTP. Connection state moves to `connected` .

The next four sections unpack steps 2 to 6.

Minimal browser-side sketch (for orientation only)

```
const pc = new RTCPeerConnection({ iceServers: [{ urls:
'stun:stun.l.google.com:19302' }] });
const stream = await navigator.mediaDevices.getUserMedia({ audio: true,
video: true });
stream.getTracks().forEach((t) => pc.addTrack(t, stream));

pc.onicecandidate = (e) => e.candidate && signaling.send({ type: 'candidate',
candidate: e.candidate });
pc.ontrack = (e) => (remoteVideo.srcObject = e.streams[0]);

const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
signaling.send({ type: 'offer', sdp: pc.localDescription });
```

That is the whole shape of a browser-to-browser call. mediasoup and LiveKit keep the same ideas but move the offer/answer work behind their own APIs, so you rarely touch raw SDP yourself.

2. Signaling and SDP

Signaling is the out-of-band conversation that sets up a connection. WebRTC does not define the transport or the message format, so any channel works: WebSocket, Socket.IO, HTTP polling, even copy-paste. Signaling carries three kinds of message: session descriptions (SDP offer and answer), ICE candidates, and your own application events (join, leave, mute).

What signaling must do

- Let peers discover each other (rooms, user ids).
- Relay SDP offers and answers between exactly the two sides of a connection.
- Relay ICE candidates as they are found.
- Carry application events: who joined, who is producing which track, who left.

Signaling is not secure by itself. Authenticate the socket (a JWT in the handshake) and authorise who may join which room. Media encryption (section 4) does not protect your signaling channel.

SDP (Session Description Protocol)

SDP is a plain-text description of a media session. It is not a WebRTC invention; WebRTC reuses it. An SDP blob has a session section and one **media section** (an "m-line") per audio, video or data stream.

```
v=0
o=- 4611731400430051336 2 IN IP4 127.0.0.1
s=-
t=0 0
a=group:BUNDLE 0 1
m=audio 9 UDP/TLS/RTP/SAVPF 111
c=IN IP4 0.0.0.0
a=mid:0
a=sendrecv
a=rtpmap:111 opus/48000/2
a=fmtp:111 minptime=10;useinbandfec=1
a=ice-ufrag:Ab3d
a=ice-pwd:...
a=fingerprint:sha-256 AA:BB:...
a=setup:actpass
m=video 9 UDP/TLS/RTP/SAVPF 96 97
a=mid:1
a=sendrecv
a=rtpmap:96 VP8/90000
a=rtpmap:97 H264/90000
a=rtcp-fb:96 nack
a=rtcp-fb:96 goog-remb
```

Lines worth recognising

Line	Meaning
`m=audio ... 111`	A media section; `111` is a payload type that is defined by an `rtpmap` line below
`a=rtpmap:111 opus/48000/2`	Payload type 111 means the Opus codec at 48 kHz, 2 channels
`a=sendrecv` / `sendonly` / `recvonly` / `inactive`	Direction of media for this section

Line	Meaning
`a=mid:0`	Media id: the stable name of this m-line, used to match offer and answer
`a=group:BUNDLE 0 1`	Send all media over one transport (one port) instead of one per track
`a=ice-ufrag`, `a=ice-pwd`	Credentials that ICE connectivity checks use
`a=fingerprint`	Hash of the DTLS certificate; the other side verifies it during the handshake
`a=setup:actpass`	Who plays the DTLS client or server role
`a=rtcp-fb:... nack`	Feedback mechanisms supported (retransmit requests, bandwidth estimates)

The offer / answer model

Negotiation is a two-message exchange with a strict state machine on each `RTCPeerConnection` (`signalingState`):

1. `stable`: nothing in progress.
2. Caller runs `createOffer()` then `setLocalDescription(offer)`, state becomes `have-local-offer`.
3. Callee runs `setRemoteDescription(offer)`, state becomes `have-remote-offer`, then `createAnswer()` and `setLocalDescription(answer)`, state returns to `stable`.
4. Caller runs `setRemoteDescription(answer)`, state returns to `stable`.

The answer may only select a **subset** of what the offer proposed: codecs, directions, extensions. That is how two browsers with different codec support settle on a common one.

Transceivers and renegotiation

In the modern API each m-line is backed by an `RTCRtpTransceiver`, which pairs a sender and a receiver. `addTrack()` and `addTransceiver()` create them. Whenever you add or remove a track after connecting, the connection fires `negotiationneeded` and you must run offer/answer again (**renegotiation**).

Both sides renegotiating at once causes *glare* (two offers crossing). The standard cure is the **perfect negotiation** pattern: one side is "polite" and rolls back its own offer when a collision occurs. You will not hand-write this with mediasoup or LiveKit, because they sidestep SDP renegotiation, which is a large part of why they exist.

Trickle ICE

Instead of waiting for all candidates before sending the offer, each candidate is sent as soon as it is found (`onicecandidate`). The receiver calls `addIceCandidate()`. This shaves seconds off connection setup. An empty candidate marks the end of gathering.

Check yourself

- Why can the answer never contain a codec that the offer lacked?
- What event tells you to renegotiate after adding a screen-share track?
- Where in your Express app does the SDP travel, and who validates it?

3. NAT traversal: ICE, STUN and TURN

Most devices sit behind a NAT router and have a private address such as `192.168.1.20`. A remote peer cannot connect to that address. **ICE (Interactive Connectivity Establishment)** is the procedure that finds a network path that works anyway, and it is why WebRTC connects through home routers, offices and mobile networks.

Candidates

A candidate is one transport address a peer could be reached on. ICE gathers several kinds:

Type	Where it comes from	When it works
`host`	A local interface address of the device	Both peers on the same LAN
`srflx` (server reflexive)	Your public IP and port as seen by a STUN server	Most home and office NATs
`prflx` (peer reflexive)	Learned during connectivity checks	Appears when a NAT maps differently per destination

Type	Where it comes from	When it works
`relay`	An address allocated on a TURN server	Strict NATs and firewalls; always works, costs bandwidth

STUN

STUN (Session Traversal Utilities for NAT) is a tiny protocol: you send a request to a public server, it answers "your packet came from `203.0.113.7:51234`". That answer becomes your `srflx` candidate. STUN servers are cheap and carry no media. Public ones exist, but production apps run their own.

TURN

TURN (Traversal Using Relays around NAT) is a relay. When no direct path works, both peers send media to the TURN server, which forwards it. It is the fallback that makes the connection succeed, at the price of server bandwidth, and a typical estimate is that a minority of sessions (roughly 10 to 20 percent, higher on corporate networks) need it. Treat that figure as approximate; measure your own users.

TURN options worth knowing:

- **UDP** on port 3478 is the default and the fastest.
- **TCP** on 3478 and **TLS** on 443 (`turns:`) get through firewalls that block UDP. TLS on 443 looks like ordinary HTTPS traffic.
- Credentials are normally short-lived, generated by your backend (the time-limited shared-secret scheme), not hard-coded.
- Common open-source server: **coturn**.

```
const pc = new RTCPeerConnection({
  iceServers: [
    { urls: 'stun:stun.example.com:3478' },
    { urls: ['turn:turn.example.com:3478?transport=udp',
'turns:turn.example.com:443?transport=tcp'],
      username: 'user-from-your-backend', credential: 'short-lived-secret' },
  ],
});
```

How ICE picks a path

1. **Gather**: collect host, srflx and relay candidates, trickling each to the peer.
2. **Pair**: form candidate pairs (every local candidate with every remote one) and rank them. Direct beats relayed, UDP beats TCP.
3. **Check**: send STUN binding requests on each pair using the `ice-ufrag` and `ice-pwd` from the SDP. A pair that gets a response is valid.
4. **Nominate**: the controlling side picks the best valid pair, and media starts there.
5. **Keep alive**: periodic checks continue; if the path dies, ICE can restart on another pair.

Connection states (`pc.iceConnectionState`): `new` , `checking` , `connected` , `completed` , `disconnected` , `failed` , `closed` . `disconnected` can recover on its own; `failed` needs an **ICE restart** (`pc.restartIce()` or `createOffer({ iceRestart: true })`).

ICE with a server in the middle

With an SFU the "remote peer" is your media server, which has a public IP. It usually runs in **ICE-lite** mode: it only answers checks and does not gather candidates through STUN. It still needs its public address advertised correctly, which is the single most common cause of "it works on my laptop but not in the cloud" (section 9).

Check yourself

- Which candidate type proves your peer is reachable only through a relay?
- Why is TURN on port 443 over TLS valuable in enterprise networks?
- What does `failed` mean, and what is the fix?

4. Security and the media pipeline

Once ICE has a working path, three layers take over: a handshake that sets up encryption, a media transport that carries timed packets, and feedback loops that keep quality acceptable on a changing network.

Encryption is mandatory

WebRTC refuses to send unencrypted media. Two protocols cooperate:

- **DTLS** (Datagram TLS) runs over the ICE path. Each side proves it owns the certificate whose hash appeared in the SDP `a=fingerprint` line, and they derive shared keys. If the fingerprint does not match, the connection is rejected. This is why signaling must

be trustworthy: someone who can alter the SDP can swap fingerprints.

- **SRTP** (Secure RTP) encrypts the audio and video packets using keys from the DTLS handshake (DTLS-SRTP). Data channels run SCTP inside DTLS.

With an SFU, encryption is **hop by hop**: browser to server, then server to browser. The server decrypts packets to route them. End-to-end encryption through an SFU needs an extra layer (insertable streams or frame encryption), which both mediasoup and LiveKit can support but neither enables by default.

RTP and RTCP

RTP carries media. Each packet has a payload type (the codec), a sequence number (ordering and loss detection), a timestamp (playout timing) and an **SSRC** (a 32-bit id of the stream). **RTCP** is the control channel that travels alongside it and carries statistics and feedback.

RTCP message	Meaning	Reaction
Sender / Receiver Report	Packets sent, lost, jitter, round-trip time	Quality stats
NACK	"I lost packet N"	Sender retransmits it
PLI / FIR	"I cannot decode; send a full keyframe"	Encoder produces a keyframe
REMB / transport-cc	Receiver-side bandwidth estimate / per-packet arrival feedback	Sender adapts its bitrate

Codecs

Codec	Kind	Notes
Opus	Audio	Mandatory in browsers; 6 to 510 kbit/s; built-in loss concealment and optional in-band FEC
VP8	Video	Mandatory, royalty-free, widely supported, supports simulcast everywhere
H.264	Video	Mandatory; hardware encode on many phones and laptops; profile mismatches cause interop bugs

Codec	Kind	Notes
VP9	Video	Better compression; supports SVC; heavier on CPU
AV1	Video	Best compression, supports SVC; encoder support still uneven across devices

The codecs your server accepts are configured on the server (the mediasoup `mediaCodecs` list, LiveKit's room and publish options), and negotiation must find an overlap with each browser.

Staying alive on a bad network

- **Jitter buffer**: the receiver holds packets briefly to smooth uneven arrival. Larger buffers mean smoother video and higher latency.
- **Congestion control**: Google Congestion Control (GCC) estimates available bandwidth from delay and loss, and the sender lowers or raises its bitrate. This is why video gets blurry before it freezes.
- **Loss repair**: NACK retransmission, forward error correction, and keyframe requests.
- **Adaptation**: the browser can lower resolution or frame rate (`degradationPreference`), and with simulcast the server can switch the layer it forwards (section 5).

Data channels

`RTCDataChannel` offers WebSocket-like messaging over SCTP over DTLS. Options per channel: `ordered` or unordered, `maxRetransmits` or `maxPacketLifeTime` (partial reliability), which is why games use unreliable unordered channels for position updates. mediasoup supports data producers and consumers; LiveKit offers reliable and lossy data publishing.

```
const dc = pc.createDataChannel('state', { ordered: false, maxRetransmits: 0
});
dc.onopen = () => dc.send(JSON.stringify({ x: 10, y: 4 }));
```

Observability

`pc.getStats()` returns the real numbers behind every behaviour above: packets lost, round-trip time, jitter, frames decoded and dropped, the selected candidate pair, bitrate. Chrome's `chrome://webrtc-internals` graphs them live, and you should keep it open

while testing every practical step.

Check yourself

- What would an attacker who controls signaling be able to do to a DTLS-SRTP call?
- A viewer's video freezes, then jumps back to normal: which RTCP message was probably involved?
- Why does an SFU make end-to-end encryption harder than a mesh call?

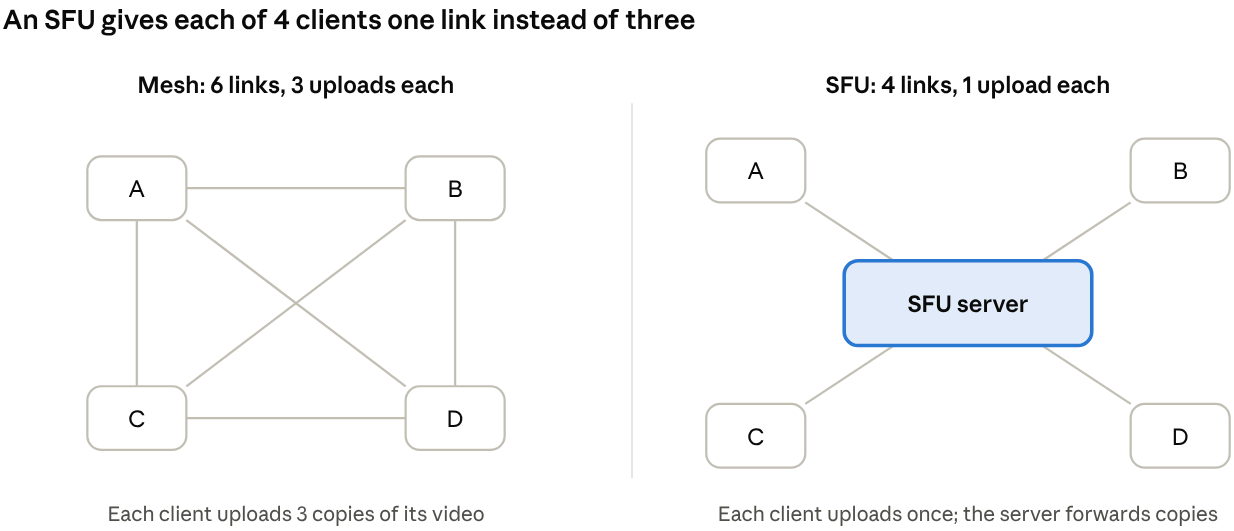
5. Topologies: mesh, MCU and SFU

A two-person call is easy. The hard question is what happens with five, fifty or five thousand participants. Three architectures answer it differently, and the choice drives every practical decision later.

	Mesh	MCU	SFU
Idea	Every peer connects to every other peer	Server decodes everyone, mixes one composite stream per viewer	Server forwards encrypted streams without decoding
Uplink per client	N-1 copies of your video	1	1 (or 1 per simulcast layer)
Downlink per client	N-1 streams	1 mixed stream	N-1 streams, but each can be a chosen quality
Server CPU	None	Very high (decode and encode)	Low (packet routing)
Latency	Lowest	Highest (transcoding)	Low
Layout control	Client	Server (fixed composite)	Client
Practical size	2 to 4 people	Legacy and SIP bridging, recording	10s to 1000s per room

The SFU won the market for conferencing because it keeps latency low and server cost moderate while leaving layout and rendering to the client. Both mediasoup and LiveKit are SFUs. Recording, streaming to HLS, and phone-dial-in are usually built by attaching a

special participant (an "egress" or a plain RTP consumer) to the SFU rather than by building an MCU.



mesh vs SFU · 4 clients

With four clients a mesh needs 6 connections and each client uploads its video three times; an SFU needs 4 connections and one upload per client.

What an SFU actually does

1. Each participant **publishes** its tracks to the server over one connection (the uplink).
2. The server keeps a **forwarding table**: for each subscriber and each published track, whether and how to forward it.
3. Each participant **subscribes** to the tracks it wants over one connection (the downlink). Packets are rewritten (sequence numbers, timestamps, SSRC) so the subscriber sees a clean continuous stream even when the server switches sources or layers.
4. The server relays RTCP feedback, so a subscriber's `PLI` reaches the right publisher.

In mediasoup terms, the publish side is a **Producer** and the subscribe side is a **Consumer**. LiveKit uses the same idea with *published tracks* and *subscriptions*.

Simulcast and SVC

A publisher with a good connection should not force a subscriber on a bad one to receive the same bitrate. Two techniques solve that without transcoding:

- **Simulcast**: the publisher encodes the *same* video at several qualities at once (for example 1280x720, 640x360, 320x180) and sends all of them. The SFU forwards the one that fits each subscriber's bandwidth and tile size. Costs publisher CPU and uplink; works with VP8 and H.264.
- **SVC (Scalable Video Coding)**: the publisher sends *one* stream built of layers (spatial, temporal); the SFU drops upper layers for weak subscribers. More efficient than simulcast; needs VP9 or AV1 and good encoder support.

Both let the server apply **dynamic broadcasting rules**: show the active speaker in high quality, others as thumbnails, pause video for off-screen tiles.

Which one should you pick?

- Learning how WebRTC works, or a one-to-one call: peer to peer with a tiny signaling server.
- A custom product that needs full control of transports, routing and scaling: **mediasoup** (a library you embed in Node).
- A product that needs rooms, auth, recording, SDKs for every platform and horizontal scaling soon: **LiveKit** (a ready SFU you operate or rent, controlled through an API).

Check yourself

- Why does a mesh of 8 people strain a laptop uplink?
- What does a simulcast-capable SFU do when a subscriber's bandwidth drops?
- Why does an SFU not need to decode video, and what breaks if it must change a participant's layout?

6. Project setup and first browser steps

One repository holds both labs, so you can compare them side by side. Everything is ES modules (`"type": "module"`), Express on the server, and esbuild to bundle browser code that imports npm packages.

```
webrtc-lab/
  package.json
  .env
  server/
    index.js              Express app, static files, mounts both labs
    mediasoup/            Lab A: worker, router, signaling
    livekit/              Lab B: token and webhook routes
  client/
    preview.js            Step 6.3
    mediasoup.js          Lab A browser code
    livekit.js            Lab B browser code
  public/
    index.html, mediasoup.html, livekit.html
    *.bundle.js           generated by esbuild
```

Step 6.1 Create the project

- **Aim**: a Node project that can run an Express server and bundle browser code.
- **Input / Output**: nothing in, a folder with `package.json` and installed packages out.
- **Skeleton**:

```
mkdir webrtc-lab && cd webrtc-lab
npm init -y            # then set "type": "module" in package.json
npm i express socket.io dotenv
npm i -D esbuild nodemon
```

- **Check**: `node -v` prints 20 or higher, and `package.json` lists the packages.

Step 6.2 Express server that serves `public/`

- **Aim**: one HTTP server for static pages now, and for signaling and API routes later.
- **Input / Output**: `GET /` in, `public/index.html` out; `GET /health` in, `{ ok: true }` out.
- **Skeleton** (`server/index.js`):

```
import express from 'express';
import http from 'node:http';
import 'dotenv/config';

const app = express();
app.use(express.json());
app.use(express.static('public'));
app.get('/health', (req, res) => res.json({ ok: true }));

const httpServer = http.createServer(app);   // Socket.IO attaches to this in
Lab A
httpServer.listen(process.env.PORT || 3000, () => console.log('listening'));

export { app, httpServer };
```

- **Check**: `node server/index.js`, then open `http://localhost:3000/health` and see `{"ok":true}`.

Step 6.3 Camera and microphone preview

- **Aim**: prove that capture works and learn the secure-context rule before adding any networking.
- **Input / Output**: a click in, a live `<video>` element out, plus a printed list of devices and tracks.
- **Skeleton** (`client/preview.js`):

```
const video = document.querySelector('#local');
document.querySelector('#start').onclick = async () => {
  const stream = await navigator.mediaDevices.getUserMedia({
    audio: { echoCancellation: true, noiseSuppression: true },
    video: { width: { ideal: 1280 }, height: { ideal: 720 }, frameRate: {
ideal: 30 } },
  });
  video.srcObject = stream;      // TODO: set video.muted = true, and call
video.play()
  // TODO: log stream.getTracks().map(t => [t.kind, t.label,
t.getSettings()])
  // TODO: list devices with navigator.mediaDevices.enumerateDevices()
};
```

- **Check**: the browser asks for permission, you see yourself, and the console shows one audio and one video track with their real resolution and frame rate.
- **Rules to remember**: `getUserMedia` works only on `https://` or `localhost`; mute the *local* preview to avoid feedback; autoplay policies require a user click before remote audio will play.

Step 6.4 Bundle browser code

- **Aim**: let browser code `import` from npm packages (`mediasoup-client`, `livekit-client`) without a framework.
- **Input / Output**: `client/*.js` in, `public/*.bundle.js` out.
- **Skeleton** (`package.json` scripts):

```
"scripts": {
  "dev": "nodemon server/index.js",
  "build": "esbuild client/preview.js client/mediasoup.js client/livekit.js --bundle --outdir=public --out-extension:.js=.bundle.js",
  "watch": "npm run build -- --watch"
}
```

- **Check**: `npm run build` creates files in `public/`, and `index.html` with `<script src="preview.bundle.js"></script>` shows the camera preview.

Step 6.5 Read your own stats

- **Aim**: get comfortable with `chrome://webrtc-internals` before the real labs, because it is your main debugging tool.
- **Input / Output**: a loopback `RTCPeerConnection` pair in the same page in, a graph of bitrate and frames out.
- **Skeleton**: create `pcA` and `pcB`, add the tracks to `pcA`, exchange offer, answer and candidates through plain function calls (no network), and set `pcB.ontrack` to a second `<video>`.
- **Check**: open `chrome://webrtc-internals` in another tab; you see two peer connections, a selected candidate pair of type `host`, and non-zero `bytesSent` growing.

When all five steps pass, you have verified capture, serving and bundling, so any failure later is about WebRTC itself and not your setup.

7. Lab A: build an SFU with mediasoup

mediasoup is a Node.js library that wraps a C++ media worker. It gives you **no signaling, no rooms and no auth**: it is deliberately low level, and you write those parts in Express and Socket.IO. That is exactly why it teaches WebRTC so well.

The five objects

Object	Lives on	Meaning
`Worker`	Server	A C++ subprocess that does the packet work. Create about one per CPU core
`Router`	Server	The media context of one room. It holds the codec list and routes packets between its transports. Rooms map to routers
`WebRtcTransport`	Server and client	One ICE+DTLS connection between a browser and the router. Typically each client has one send transport and one receive transport
`Producer`	Server and client	One media track (audio or video) flowing from a client into the router
`Consumer`	Server and client	One media track flowing from the router to a client, created *for a specific producer and a specific receiver*

Note the key difference from raw WebRTC: the **server** decides what to send to whom. A client never gets media unless your code creates a Consumer for it.

The signaling protocol you will implement

You define these Socket.IO events. Every request uses an acknowledgement callback so the client can `await` the answer.

Direction	Event	Payload	Reply
client to server	`join`	`{ roomId }`	`{ routerRtpCapabilities, existingProducers }`

Direction	Event	Payload	Reply
client to server	`createWebRtcTransport`	`{ direction: 'send' or 'recv' }`	`{ id, iceParameters, iceCandidates, dtlsParameters }`
client to server	`connectTransport`	`{ transportId, dtlsParameters }`	`{}`
client to server	`produce`	`{ transportId, kind, rtpParameters, appData }`	`{ id }`
client to server	`consume`	`{ producerId, rtpCapabilities }`	`{ id, producerId, kind, rtpParameters }`
client to server	`resumeConsumer`	`{ consumerId }`	`{}`
server to client	`newProducer`	`{ producerId, peerId, kind }`	none
server to client	`producerClosed` / `peerLeft`	`{ producerId }` / `{ peerId }`	none

The sequence for one user joining and publishing is: **join, create send transport, connect it, produce, then (for each other user's producer) create recv transport once, connect it, consume, resume.**

Install for this lab:

```
npm i mediasoup mediasoup-client
```

The server package downloads or builds a native worker on install. If the install fails, check that you have Python 3 and a C++ toolchain, or use a Linux or WSL2 machine. The server needs Node 18 or newer in current versions.

Step 7.1 Start a Worker and a Router

- **Aim**: have a running media engine and one room's router that knows which codecs it accepts.
- **Input / Output**: nothing in; a `router` object and its `router.rtpCapabilities` out.

- **Skeleton** (`server/mediasoup/engine.js`):

```
import * as mediasoup from 'mediasoup';

const mediaCodecs = [
  { kind: 'audio', mimeType: 'audio/opus', clockRate: 48000, channels: 2 },
  { kind: 'video', mimeType: 'video/VP8', clockRate: 90000,
    parameters: { 'x-google-start-bitrate': 1000 } },
  // TODO: add video/H264 (packetization-mode 1, profile-level-id 42e01f) for
Safari and phones
];

export async function startWorker() {
  const worker = await mediasoup.createWorker({
    logLevel: 'warn',
    rtcMinPort: 40000, rtcMaxPort: 40100,   // UDP ports the worker may use
  });
  worker.on('died', () => { console.error('worker died'); process.exit(1);
});
  return worker;
}

export async function createRoomRouter(worker) {
  return worker.createRouter({ mediaCodecs });
}
```

- **Check**: log `router.rtpCapabilities.codecs.map(c => c.mimeType)` and see `audio/opus` and `video/VP8` (mediasoup also lists internal RTX entries for retransmission).
- **Note**: newer mediasoup versions move the port range into each transport's `listenInfos` (`portRange`) and add a `WebRtcServer` that lets many transports share one port. The worker-level range above is fine for a lab; read the changelog of your installed version before production.

Step 7.2 Room registry and the signaling skeleton

- **Aim**: track who is in which room, and own every transport, producer and consumer so you can clean them up.
- **Input / Output**: a Socket.IO connection and a `join` event in; the router capabilities and the list of producers already in the room out.
- **Skeleton** (`server/mediasoup/signaling.js`):

```
import { Server } from 'socket.io';
import { startWorker, createRoomRouter } from './engine.js';

const rooms = new Map();   // roomId -> { router, peers: Map(peerId -> peer)
}
// peer = { socket, transports: Map, producers: Map, consumers: Map }

export async function attachSignaling(httpServer) {
  const worker = await startWorker();
  const io = new Server(httpServer);

  io.on('connection', (socket) => {
    let roomId, peer;

    socket.on('join', async (data, cb) => {
      roomId = data.roomId;
      if (!rooms.has(roomId)) rooms.set(roomId, { router: await
createRoomRouter(worker), peers: new Map() });
      const room = rooms.get(roomId);
      peer = { socket, transports: new Map(), producers: new Map(),
consumers: new Map() };
      room.peers.set(socket.id, peer);
      socket.join(roomId);
      cb({
        routerRtpCapabilities: room.router.rtpCapabilities,
        existingProducers: [], // TODO: list { producerId, peerId, kind } of
every producer of other peers
      });
    });

    // TODO Steps 7.3 to 7.6: createWebRtcTransport, connectTransport,
produce, consume, resumeConsumer
    // TODO Step 7.8: disconnect cleanup
  });
}
```

- **Wire it up**: in `server/index.js`, call `attachSignaling(httpServer)` before `listen`.
- **Check**: connect from the browser console with a Socket.IO client, emit `join` with `{ roomId: 'a' }`, and see the capabilities object in the callback.

What comes next

Steps 7.3 to 7.6 build the server half of transports, producing and consuming. Steps 7.7 to 7.10 build the browser half, add cleanup and test everything.

Step 7.3 Create a WebRTC transport on request

- **Aim**: give the browser everything it needs to open an ICE and DTLS connection to the router.
- **Input / Output**: `{ direction }` in; `{ id, iceParameters, iceCandidates, dtlsParameters }` out. These four values are what a browser would normally learn from an SDP answer, now delivered as plain JSON.
- **Skeleton** (inside the `connection` handler, after `join`):

```
socket.on('createWebRtcTransport', async ({ direction }, cb) => {
  const room = rooms.get(roomId);
  const transport = await room.router.createWebRtcTransport({
    listenInfos: [
      { protocol: 'udp', ip: '0.0.0.0', announcedAddress:
process.env.ANNOUNCED_IP || '127.0.0.1' },
      { protocol: 'tcp', ip: '0.0.0.0', announcedAddress:
process.env.ANNOUNCED_IP || '127.0.0.1' },
    ],
    enableUdp: true, enableTcp: true, preferUdp: true,
    appData: { direction },
  });
  peer.transports.set(transport.id, transport);
  // TODO: transport.on('dtlsstatechange', (s) => s === 'closed' &&
transport.close());
  cb({
    id: transport.id,
    iceParameters: transport.iceParameters,
    iceCandidates: transport.iceCandidates,
    dtlsParameters: transport.dtlsParameters,
  });
});
```

- **Check**: the reply contains `iceCandidates` whose `ip` equals your `ANNOUNCED_IP`. For a lab on one machine use `127.0.0.1`; to test from a phone on your Wi-Fi use your computer's LAN address.

- **Why the announced address**: the worker listens on `0.0.0.0` but must tell browsers a concrete address to send packets to. A wrong value here is the number one reason for a transport that never connects. Older mediasoup versions call this field `announcedIp` and use a `listenIps` option, so match the names to your installed version.

Step 7.4 Finish the DTLS handshake

- **Aim**: let the server learn the browser's DTLS fingerprint so encryption can start.
- **Input / Output**: `{ transportId, dtlsParameters }` in; `{}` out. The browser sends this automatically from its transport's `connect` event (Step 7.7).
- **Skeleton**:

```
socket.on('connectTransport', async ({ transportId, dtlsParameters }, cb) =>
{
  const transport = peer.transports.get(transportId);
  // TODO: reply with { error } when the transport is missing
  await transport.connect({ dtlsParameters });
  cb({});
});
```

- **Check**: after the browser step works, `transport.dtlsState` becomes `connected` and the console shows ICE state `completed` or `connected`.

Step 7.5 Accept a published track (Producer)

- **Aim**: receive one audio or video track from a client and tell everyone else it exists.
- **Input / Output**: `{ transportId, kind, rtpParameters, appData }` in; `{ id }` out, plus a `newProducer` event broadcast to the rest of the room.
- **Skeleton**:

```
socket.on('produce', async ({ transportId, kind, rtpParameters, appData },
cb) => {
  const transport = peer.transports.get(transportId);
  const producer = await transport.produce({ kind, rtpParameters, appData });
  peer.producers.set(producer.id, producer);
  producer.on('transportclose', () => peer.producers.delete(producer.id));
  socket.to(roomId).emit('newProducer', { producerId: producer.id, peerId:
socket.id, kind });
  cb({ id: producer.id });
});
```

- **Also finish Step 7.2**: fill `existingProducers` by looping over the other peers in the room and collecting `{ producerId, peerId, kind }` from each `peer.producers`, so late joiners see people already publishing.
- **Check**: with the browser step in place, the server logs one producer per track, and `await producer.getStats()` shows a growing `byteCount`.

Step 7.6 Send a track to a viewer (Consumer)

- **Aim**: create the per-viewer copy of a producer and hand the browser the parameters to receive it.
- **Input / Output**: `{ producerId, rtpCapabilities, transportId }` in (the browser's `device.rtpCapabilities` and its receive transport); `{ id, producerId, kind, rtpParameters }` out.
- **Skeleton**:

```
socket.on('consume', async ({ producerId, rtpCapabilities, transportId }, cb)
=> {
  const room = rooms.get(roomId);
  if (!room.router.canConsume({ producerId, rtpCapabilities })) return cb({
error: 'cannot consume' });

  const transport = peer.transports.get(transportId);
  const consumer = await transport.consume({ producerId, rtpCapabilities,
paused: true });
  peer.consumers.set(consumer.id, consumer);
  consumer.on('producerclose', () => {
    socket.emit('producerClosed', { consumerId: consumer.id });
    consumer.close(); peer.consumers.delete(consumer.id);
  });
  cb({ id: consumer.id, producerId, kind: consumer.kind, rtpParameters:
consumer.rtpParameters });
});

socket.on('resumeConsumer', async ({ consumerId }, cb) => {
  await peer.consumers.get(consumerId).resume();
  cb({});
});
```

- **Why `paused: true`**: a video consumer needs a keyframe to start. Creating it paused, letting the browser build its side, and then resuming guarantees the browser is ready when the first keyframe arrives, so you avoid a black first second.

- **Why `canConsume`**: it checks that the viewer's codecs overlap with what the producer sends. With only VP8 and Opus configured this nearly always passes, which changes the moment you add H.264 or VP9.
- **Check**: the reply contains `rtpParameters.codecs` with a payload type and an SSRC in `encodings`.

Step 7.7 Browser: load the device, connect a send transport, publish

- **Aim**: make the browser speak the mediasoup protocol and send its camera and microphone to the router.
- **Input / Output**: a click in; two producers (audio, video) on the server out, and your own preview on screen.
- **Setup**: `npm i socket.io-client`, then write `client/mediasoup.js` and rebuild with esbuild.
- **Skeleton**:

```
import { io } from 'socket.io-client';
import { Device } from 'mediasoup-client';

const socket = io();
const request = (event, data) => new Promise((resolve) => socket.emit(event,
data, resolve));

const device = new Device();
let sendTransport, recvTransport;

async function join(roomId) {
  const { routerRtpCapabilities, existingProducers } = await request('join',
{ roomId });
  await device.load({ routerRtpCapabilities });
  // TODO: store existingProducers for Step 7.9
}

async function createSendTransport() {
  const params = await request('createWebRtcTransport', { direction: 'send'
});
  sendTransport = device.createSendTransport(params);   // { id,
iceParameters, iceCandidates, dtlsParameters }

  sendTransport.on('connect', ({ dtlsParameters }, callback, errback) => {
    request('connectTransport', { transportId: sendTransport.id,
```

```
dtlsParameters }).then(callback).catch(errback);
  });

  sendTransport.on('produce', ({ kind, rtpParameters, appData }, callback,
errback) => {
    // TODO: request('produce', { transportId: sendTransport.id, kind,
rtpParameters, appData })
    //       then call callback({ id }) with the id the server returns
  });
}

async function publish() {
  const stream = await navigator.mediaDevices.getUserMedia({ audio: true,
video: true });
  // TODO: show the local preview (muted)
  await sendTransport.produce({ track: stream.getAudioTracks()[0] });
  await sendTransport.produce({ track: stream.getVideoTracks()[0] });
}
```

- **Check**: after `join()`, `createSendTransport()`, `publish()`: the server logs two producers, the transport's `connectionState` goes `new`, `connecting`, `connected` (listen with `sendTransport.on('connectionstatechange', ...)`), and `chrome://webrtc-internals` shows one peer connection sending two tracks.
- **How it maps to raw WebRTC**: the `connect` event is the DTLS part of the answer; `produce` replaces the offer. There is no SDP in your code; `mediasoup-client` builds it privately and hands you only the parameters that matter.

Step 7.8 Server: clean up when a peer leaves

- **Aim**: free every transport, producer and consumer a peer owned, and tell the others, so rooms do not leak memory or leave frozen tiles.
- **Input / Output**: the Socket.IO `disconnect` event in; a `peerLeft` event to the room and an empty registry entry out.
- **Skeleton** (inside the `connection` handler):

```
socket.on('disconnect', () => {
  const room = rooms.get(roomId);
  if (!room || !peer) return;
  peer.transports.forEach((t) => t.close());   // closing a transport closes
its producers and consumers
  room.peers.delete(socket.id);
  socket.to(roomId).emit('peerLeft', { peerId: socket.id });
  // TODO: when room.peers.size === 0, close room.router and delete the room
});
```

- **Check**: close one tab. The other tab receives `peerLeft` (and `producerClosed` for each consumer), and the server's room map shrinks. Log the number of rooms and peers on every change while you develop.

Step 7.9 Browser: create a receive transport and consume

- **Aim**: show every other participant's audio and video.
- **Input / Output**: a producer id in (from `existingProducers` or a `newProducer` event); a `<video>` or `<audio>` element playing that track out.
- **Skeleton**:

```
async function createRecvTransport() {
  const params = await request('createWebRtcTransport', { direction: 'recv'
});
  recvTransport = device.createRecvTransport(params);
  recvTransport.on('connect', ({ dtlsParameters }, callback, errback) => {
    // TODO: same as the send transport, using recvTransport.id
  });
}

async function consume(producerId, peerId) {
  if (!recvTransport) await createRecvTransport();
  const data = await request('consume', {
    producerId, rtpCapabilities: device.rtpCapabilities, transportId:
recvTransport.id,
  });
  if (data.error) return console.warn(data.error);

  const consumer = await recvTransport.consume({
    id: data.id, producerId: data.producerId, kind: data.kind, rtpParameters:
data.rtpParameters,
```

```
  });
  const stream = new MediaStream([consumer.track]);
  // TODO: find or create the element for peerId, set element.srcObject =
stream, element.play()
  await request('resumeConsumer', { consumerId: consumer.id });
}

socket.on('newProducer', ({ producerId, peerId }) => consume(producerId,
peerId));
socket.on('producerClosed', ({ consumerId }) => { /* TODO: remove that
element */ });
socket.on('peerLeft', ({ peerId }) => { /* TODO: remove that peer's tile */
});
```

- **Check**: open the page in two tabs, join the same room in both, and each tab shows the other's video and plays its audio. Use headphones, or mute the preview, to avoid echo.
- **Race to avoid**: two `consume` calls at once can create two receive transports. Create the transport once (for example store the promise) before the first `consume`.

Step 7.10 Test, troubleshoot, then extend

Test plan

1. Two tabs on the same machine: audio and video both ways.
2. Late joiner: open a third tab after the first two are streaming and confirm it sees both (this exercises `existingProducers`).
3. Leave and rejoin: close a tab and reopen; no frozen tiles remain.
4. Another device: set `ANNOUNCED_IP` to your LAN address, open the server through that address. Camera access on a non-localhost address needs HTTPS (use a tunnel or a local certificate).

Common failures

Symptom	Likely cause	Fix
Transport stuck on `connecting`	Wrong announced address, blocked UDP, or Docker without published ports	Check `iceCandidates` in the reply, open or publish the worker port range, test with TCP enabled

Symptom	Likely cause	Fix
Black video tile	Consumer never resumed, or element not playing	Call `resumeConsumer`, set `autoplay` and `playsInline`, call `play()` after a click
No audio from others	Browser autoplay policy	Start playback from a user gesture
`canConsume` returns false	Codec mismatch between router and device	Add the codec to `mediaCodecs`; check `device.rtpCapabilities`
`Device not loaded`	`produce` or transport creation before `device.load()`	Await `join()` first
Works on localhost, fails on phone	HTTPS needed for `getUserMedia`, or announced address is `127.0.0.1`	Serve over HTTPS, use the LAN address

Extensions, in order of value

- **Simulcast**: pass `encodings: [{ maxBitrate: 100000 }, { maxBitrate: 300000 }, { maxBitrate: 900000 }]` and `codecOptions` to the video `produce()` call, then on the server use `consumer.setPreferredLayers({ spatialLayer, temporalLayer })` to pick what each viewer gets.
- **Active speaker**: create an `AudioLevelObserver` on the router and tell clients who is talking.
- **Scale beyond one core**: one router lives in one worker, so spread rooms across workers, and use `router.pipeToRouter()` to connect routers across workers or machines.
- **Recording or RTMP**: create a `PlainTransport` and consume into FFmpeg.
- **Authentication**: verify a JWT in `io.use((socket, next) => ...)` and check it grants access to `roomId` before `join`.

8. Lab B: LiveKit with Express

LiveKit is an open-source SFU that you run yourself or use as a hosted cloud. Unlike mediasoup it is a **complete server**: it already has signaling, rooms, authentication, simulcast, reconnection and active-speaker detection. Your Express app shrinks to three jobs: issue tokens, manage rooms through an API, and receive webhooks.

The concepts

Concept	Meaning	mediasoup equivalent
LiveKit server	The SFU process, listening for WebSocket signaling and media	Worker + router + your signaling code
Room	A named session; created automatically on first join or by the API	Router (plus your room map)
Participant	One connected user, identified by a string `identity`	Your `peer` object
Track and publication	A published audio or video stream, with a `sid`	Producer
Subscription	A participant receiving a published track	Consumer
Access token	A signed JWT that says who you are and what you may do	Your own Socket.IO auth
Transports	Handled inside the SDK; you never touch them	`WebRtcTransport`

How the pieces talk

1. The browser asks **your Express server** for a token (`POST /livekit/token`).
2. Express signs a JWT with the API key and secret, granting access to one room.
3. The browser connects to the **LiveKit server** with that token. Everything after (signaling, ICE, publishing, subscribing) happens between the SDK and LiveKit, not through Express.
4. LiveKit calls your Express **webhook** when rooms start and end, participants join or leave, and tracks publish.

5. Express can call LiveKit's **server API** to list rooms, remove a participant or mute a track.

The API secret never leaves your server. A leaked secret lets anyone mint tokens for any room.

Step 8.1 Run a LiveKit server

- **Aim**: have a local SFU to talk to, with known credentials.
- **Input / Output**: a command in; a server on `ws://localhost:7880` out.
- **Skeleton**: install the `livekit-server` binary (the LiveKit docs list Homebrew, a install script and a Docker image), then run it in development mode:

```
livekit-server --dev      # dev mode prints or uses a fixed key pair: devkey
/ secret
```

Add these to `.env`:

```
LIVEKIT_URL=ws://localhost:7880
LIVEKIT_API_KEY=devkey
LIVEKIT_API_SECRET=secret
```

- **Check**: opening `http://localhost:7880` in a browser gives a short "OK" response. If you use Docker, publish the signaling port (7880), the TCP media port (7881) and the UDP port range, otherwise signaling connects but media never flows. Dev mode credentials are for local use only; production uses your own key pair in the config file.

Step 8.2 Token endpoint in Express

- **Aim**: let an authenticated user obtain permission to join one room, and nothing more.
- **Input / Output**: `{ room, identity, name }` in (the user's identity should come from *your* session, not the request body, in a real app); `{ token, url }` out.
- **Skeleton** (`server/livekit/routes.js`):

```
import { Router } from 'express';
import { AccessToken } from 'livekit-server-sdk';

export const livekitRoutes = Router();

livekitRoutes.post('/token', async (req, res) => {
  const { room, identity, name } = req.body;
  // TODO: reject missing or oversized values; in production take identity
from your auth session

  const at = new AccessToken(process.env.LIVEKIT_API_KEY,
process.env.LIVEKIT_API_SECRET, {
    identity, name, ttl: '10m',
  });
  at.addGrant({ roomJoin: true, room, canPublish: true, canSubscribe: true
});

  res.json({ token: await at.toJwt(), url: process.env.LIVEKIT_URL });   //
toJwt() is async in server SDK v2
});
```

- **Wire it up**: in `server/index.js`, `app.use('/livekit', livekitRoutes)`.
- **Check**: `curl -X POST localhost:3000/livekit/token -H 'content-type: application/json' -d '{"room":"a","identity":"sam"}'` returns a token. Paste it into a JWT decoder: the payload has `sub: sam` and a `video` claim with `roomJoin: true` and `room: a`.
- **Why short expiry**: the token only has to survive until the client connects. After that the session continues, and the SDK can fetch a new token on reconnect if you design for it.

Step 8.3 Browser: connect, publish, render everyone

- **Aim**: join a room, publish camera and microphone, and show every remote participant's tracks.
- **Input / Output**: a click in; local and remote tiles out.
- **Setup**: `npm i livekit-client`, write `client/livekit.js`, rebuild with esbuild.
- **Skeleton**:

```
import { Room, RoomEvent, Track } from 'livekit-client';

const grid = document.querySelector('#grid');
const room = new Room({ adaptiveStream: true, dynacast: true });

room
  .on(RoomEvent.TrackSubscribed, (track, publication, participant) => {
    const el = track.attach();              // returns a <video> or <audio>
element
    // TODO: give it a data-identity attribute, append to grid
  })
  .on(RoomEvent.TrackUnsubscribed, (track) => track.detach().forEach((el) =>
el.remove()))
  .on(RoomEvent.ParticipantDisconnected, (p) => { /* TODO: remove leftovers
for p.identity */ })
  .on(RoomEvent.AudioPlaybackStatusChanged, () => { /* TODO: if
(!room.canPlaybackAudio) show an "enable audio" button that calls
room.startAudio() */ });

document.querySelector('#join').onclick = async () => {
  const { token, url } = await (await fetch('/livekit/token', {
    method: 'POST', headers: { 'content-type': 'application/json' },
    body: JSON.stringify({ room: 'lab', identity: crypto.randomUUID(), name:
'Me' }),
  })).json();

  await room.connect(url, token);
  await room.localParticipant.setCameraEnabled(true);
  await room.localParticipant.setMicrophoneEnabled(true);
  // TODO: show the local camera: attach the publication for
Track.Source.Camera, muted
};
```

- **Check**: open two tabs and join the same room. Each shows the other's video and plays audio. In `chrome://webrtc-internals` you see two peer connections per tab (one publishing, one subscribing); that split is LiveKit's design and is invisible to your code.
- **What the options do**: `adaptiveStream` lowers the requested resolution when a tile is small or hidden. `dynacast` pauses the publisher's unused simulcast layers when nobody needs them. Both save bandwidth and CPU with no extra code.

Step 8.4 Room features: speakers, mute state and data messages

- **Aim**: add the behaviour users expect from a call, using events the SDK already emits.
- **Input / Output**: SDK events in; UI changes (speaking border, mute icon, chat line) out.
- **Skeleton**:

```
room.on(RoomEvent.ActiveSpeakersChanged, (speakers) => {
  // TODO: toggle a CSS class on the tiles whose participant.identity is in
speakers
});
room.on(RoomEvent.TrackMuted, (pub, participant) => { /* TODO: show muted
icon */ });
room.on(RoomEvent.TrackUnmuted, (pub, participant) => { /* TODO: hide muted
icon */ });
room.on(RoomEvent.ConnectionQualityChanged, (quality, participant) => { /*
TODO: show bars */ });

// chat over the data channel
room.on(RoomEvent.DataReceived, (payload, participant, kind, topic) => {
  const msg = new TextDecoder().decode(payload);
  // TODO: append msg to the chat list, with participant?.identity as the
sender
});
async function sendChat(text) {
  await room.localParticipant.publishData(new TextEncoder().encode(text), {
reliable: true, topic: 'chat' });
}

// also try:
// await room.localParticipant.setScreenShareEnabled(true);
```

- **Check**: speak in one tab and the other highlights that tile; mute and see the icon; send a chat line and it appears in the other tab.

Step 8.5 Room management from Express

- **Aim**: let your backend create rooms with limits, see who is inside, and remove or mute people.
- **Input / Output**: HTTP calls to your Express routes in; JSON about rooms and participants out.
- **Skeleton** (add to `server/livekit/routes.js`):

```
import { RoomServiceClient } from 'livekit-server-sdk';

const rooms = new RoomServiceClient(
  process.env.LIVEKIT_URL.replace('ws', 'http'),     // the API uses http(s),
not ws(s)
  process.env.LIVEKIT_API_KEY, process.env.LIVEKIT_API_SECRET,
);

livekitRoutes.post('/rooms', async (req, res) => {
  const room = await rooms.createRoom({ name: req.body.name, emptyTimeout:
600, maxParticipants: 20 });
  res.json(room);
});
livekitRoutes.get('/rooms', async (req, res) => res.json(await
rooms.listRooms()));
livekitRoutes.get('/rooms/:name/participants', async (req, res) => {
  res.json(await rooms.listParticipants(req.params.name));
});
// TODO: DELETE /rooms/:name/participants/:identity ->
rooms.removeParticipant(name, identity)
// TODO: POST mute -> rooms.mutePublishedTrack(name, identity, trackSid,
true)
```

- **Check**: with two tabs connected, `GET /livekit/rooms` shows one room with `numParticipants: 2`, and removing one identity disconnects that tab.
- **Protect these routes**: they are admin actions. Put your normal auth middleware in front of them; recent SDK versions also offer a unified `LiveKitAPI` client that wraps these services, so check the reference for your version.

Step 8.6 Webhooks: react to what happens in rooms

- **Aim**: learn about rooms and participants from the server side, for example to record attendance or start a recording.
- **Input / Output**: LiveKit POSTs signed JSON in; your handler logs or stores the event, returns `200`.
- **Skeleton** (mount this *before* any global body parser that could consume the body):

```
import express from 'express';
import { WebhookReceiver } from 'livekit-server-sdk';

const receiver = new WebhookReceiver(process.env.LIVEKIT_API_KEY,
process.env.LIVEKIT_API_SECRET);

livekitRoutes.post('/webhook', express.raw({ type: 'application/webhook+json'
}), async (req, res) => {
  try {
    const event = await receiver.receive(req.body.toString(),
req.get('Authorization'));
    // event.event is e.g. 'room_started', 'participant_joined',
'track_published', 'room_finished'
    // TODO: switch on event.event and store or log event.room /
event.participant
    res.sendStatus(200);
  } catch (err) {
    res.sendStatus(401);        // bad signature: never trust unsigned
webhooks
  }
});
```

- **Tell the server where to post** with a config file (`livekit.yaml`) and run `livekit-server --config livekit.yaml` . Field names below match the documented config; confirm them against your server version.

```
port: 7880
rtc:
  tcp_port: 7881
  port_range_start: 50000
  port_range_end: 50100
  use_external_ip: false
keys:
  devkey: secret
webhook:
  api_key: devkey
  urls:
    - http://localhost:3000/livekit/webhook
```

- **Check**: join a room and watch your Express console print `room_started` and `participant_joined` ; leave and see `participant_left` , then `room_finished` after the empty timeout.

- **Gotcha**: signature verification uses the raw bytes, so a JSON parser that runs first will break it. That is why this route uses `express.raw`. If the server runs in Docker and Express on the host, use the host's reachable address (for example `host.docker.internal`) instead of `localhost`.

Step 8.7 Roles through token grants

- **Aim**: express who may publish, who may only watch and who may moderate, entirely in the token.
- **Input / Output**: a `role` your auth layer decided in; a token with different grants out.
- **Skeleton**:

```
const grants = {
  host:   { roomJoin: true, room, canPublish: true,  canSubscribe: true,
roomAdmin: true },
  guest:  { roomJoin: true, room, canPublish: true,  canSubscribe: true },
  viewer: { roomJoin: true, room, canPublish: false, canSubscribe: true,
canPublishData: false },
};
// at.addGrant(grants[role]);
```

- **Check**: a `viewer` token joins and sees everyone, but `setCameraEnabled(true)` fails with a permission error. A `host` can remove participants from the client SDK as well.
- **Rule**: the role comes from your database or session on the server, never from the request body.

Step 8.8 Troubleshoot, then go further

Symptom	Likely cause	Fix
`room.connect` rejects with an auth error	Wrong key or secret, expired token, wrong `room` in the grant	Decode the JWT, compare with server keys, check clock skew
Connects, but no video flows	UDP or TCP media ports unreachable (Docker, firewall, cloud security group)	Publish ports, enable the TCP fallback port, set the node's external IP in the config
Remote audio silent	Autoplay policy	Show a button that calls `room.startAudio()`

Symptom	Likely cause	Fix
Webhook 401	Body parsed before verification, or key mismatch	Use `express.raw`, same key as `webhook.api_key`
Token works locally, not in production	`LIVEKIT_URL` still points to `ws://localhost`	Use a `wss://` URL on a public domain with TLS

Where to go next

- **Recording and streaming**: LiveKit Egress records rooms or tracks to files and pushes to RTMP or HLS.
- **Telephony and ingest**: SIP bridges and Ingress bring phone calls, RTMP and WHIP streams into rooms.
- **Agents**: server-side participants (for example AI voice agents) that join a room like any user.
- **End-to-end encryption**: the client SDK supports an E2EE option with a shared key.
- **Scaling**: multiple LiveKit nodes coordinate through Redis, and LiveKit Cloud runs the fleet for you.

9. Compare, deploy, debug and practise

mediasoup or LiveKit?

Question	mediasoup	LiveKit
What it is	A Node library with a C++ worker that you embed	A standalone SFU server with SDKs for web, mobile and backends
Signaling, rooms, auth	You write all of it	Built in (token-based)
Client code	`mediasoup-client` plus your own protocol	`livekit-client`, one `connect` call
Control over routing	Total: you choose who consumes what, layer by layer	High through API and permissions, less raw access

Question	mediasoup	LiveKit
Mobile and native SDKs	Community and C++ client libraries	Official SDKs for iOS, Android, Flutter, React Native, Unity and more
Recording, SIP, streaming	Build it yourself (PlainTransport and FFmpeg)	Egress, Ingress and SIP services
Scaling	You design piping between workers and machines	Multi-node with Redis, or the hosted cloud
Best for	Custom media pipelines, learning, embedding into an existing Node system	Shipping a product quickly, mobile apps, teams without WebRTC specialists

A sound path: learn with mediasoup because it exposes every concept, then ship with LiveKit unless you have a reason that needs mediasoup's control.

Deployment checklist

1. **HTTPS and WSS everywhere.** Browsers refuse camera access on insecure origins, and signaling over plain `ws://` from an `https://` page is blocked.
2. **Correct public address.** mediasoup: `announcedAddress` in `listenInfos`. LiveKit: the node's external IP in its config or `use_external_ip`.
3. **Open the media ports.** mediasoup: the UDP and TCP port range or the single `WebRtcServer` port. LiveKit: 7881 TCP and the UDP range, plus 443 or TURN if you need firewall traversal. In cloud consoles this means a security-group rule, which is easy to forget.
4. **Run TURN.** Deploy coturn (or use your provider's TURN) with TLS on 443, and hand clients short-lived credentials from your backend. Test with a client on a restrictive network, not only on your own Wi-Fi.
5. **Docker.** UDP through Docker's bridge network is the usual culprit; use host networking or publish exactly the ports above.
6. **Scale by room.** mediasoup: one worker per core, rooms spread across workers, `pipeToRouter` for rooms larger than one core. LiveKit: add nodes behind the same Redis.
7. **Observe.** Record packet loss, round-trip time, bitrate and join failures per session, from `getStats()` on clients and from the server's metrics.

Bandwidth arithmetic you should do before choosing hardware. In a room of N people who each publish 1 Mbit/s, the server receives N Mbit/s and sends N × (N-1) Mbit/s if everyone watches everyone. For N = 10 that is 10 Mbit/s in and 90 Mbit/s out. Simulcast and showing only visible tiles at lower layers reduce the outbound side sharply, which is why those features matter more than raw server size.

Debugging toolbox

Tool	Use it to
`chrome://webrtc-internals`	See ICE states, selected candidate pair, bitrate, loss and frames per peer connection
`pc.getStats()` / `transport.getStats()`	Pull the same numbers into your own logs
mediasoup `logLevel: 'debug'` and `logTags`	Trace ICE, DTLS and RTP on the server
LiveKit server logs and `--dev` mode	Follow joins, publishes and subscriptions
Network throttling in browser dev tools, or OS tools	Reproduce poor networks and watch adaptation
A second device on mobile data	Reproduce NAT and TURN problems that your desk never shows

Security checklist

- Authenticate the signaling channel and authorise each room join.
- Never expose the LiveKit API secret or any mediasoup server control to the browser.
- Verify webhook signatures.
- Give tokens short lifetimes and the smallest grants that work.
- Rate-limit join and token endpoints.
- Remember encryption is hop by hop on an SFU; add end-to-end encryption if the server must not see media.

Exercises

Each exercise has a check so you know when it is done.

1. **Loopback**: connect two `RTCPeerConnection` objects in one page without any server. *Check*: remote video plays and webrtc-internals shows `host` candidates.
2. **Two-tab mesh**: add a Socket.IO signaling server and make two browsers call each other directly. *Check*: works on two devices on your Wi-Fi.
3. **Break it on purpose**: block UDP with a firewall rule and watch ICE fail, then add TURN and watch it succeed with a `relay` candidate. *Check*: the selected pair type is `relay`.
4. **mediasoup lab**: finish steps 7.1 to 7.10. *Check*: three tabs, late joiner sees everyone, closing a tab leaves no frozen tile.
5. **Simulcast**: enable three encodings on the mediasoup video producer and choose layers per consumer. *Check*: `consumer.currentLayers` changes when you throttle one viewer.
6. **LiveKit lab**: finish steps 8.1 to 8.8. *Check*: webhook events print in Express and a viewer token cannot publish.
7. **Same feature in both**: build "mute everyone" and "who is speaking" in both labs. *Check*: write down which was more code and why.
8. **Production dry run**: deploy one lab to a cloud VM with HTTPS and TURN, and test from a phone on mobile data. *Check*: video connects and `getStats` reports the candidate type you expect.

Sources and further reading

API names in the labs were checked against these references (versions move, so always read the changelog of the version you install):

- LiveKit: endpoint token generation
- LiveKit JS server SDK reference
- LiveKit JS client SDK reference
- mediasoup-client on npm
- mediasoup 3.14 release note: portRange in listenInfos
- mediasoup forum: WebRtcServer and port binding